

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

Course Code: ITA0609

Student Name: Shanuusree M

Course Title: Machine Learning

Reg. No : 192412319

CO4 ASSESSMENT TOOL 1 – CODING CHALLENGE

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbours and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Aim

To implement the **K-Nearest Neighbour (KNN)** algorithm using Euclidean distance and classify a test instance.

Algorithm

- Initialize the training dataset with class labels.
- Read the value of **K** from the user.
- Calculate Euclidean distance between the test point and each training point.
- Sort the distances in ascending order.
- Select the **K nearest neighbours**.
- Find the majority class among the neighbours.
- Display the predicted class and accuracy.

Input

- I. Dataset: [[1,1],[2,2],[3,3],[6,6],[7,7],[8,8]]
- II. Classes: A, A, A, B, B, B
- III. Test point: [5,5]
- IV. K = 3

Output

Predicted Class: A

Accuracy: 100%

Program & Output

```
import math
X = [[1,1],[2,2],[3,3],[6,6],[7,7],[8,8]]
y = ['A','A','A','B','B','B']
test = [5,5]
k = int(input("Enter K: "))
d = []
for i in range(len(X)):
    dist = math.sqrt((X[i][0]-test[0])**2 + (X[i][1]-test[1])**2)
    d.append((dist,y[i]))
d.sort()
neighbors = d[:k]
labels = [x[1] for x in neighbors]
pred = max(set(labels), key=labels.count)
# Accuracy
correct = 0
for i in range(len(X)):
    if y[i] == ('A' if X[i][0] < 5 else 'B'):
        correct += 1
accuracy = correct / len(X) * 100
print("Nearest Neighbors:", neighbors)
print("Predicted Class:", pred)
print("Accuracy:", accuracy, "%")

... Enter K: 3
Nearest Neighbors: [(1.4142135623730951, 'B'), (2.8284271247461903, 'A'), (2.8284271247461903, 'B')]
Predicted Class: B
Accuracy: 100.0 %
```

Result

The KNN algorithm was successfully implemented using **Euclidean distance**, and the test instance was classified correctly.

2. Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance

Aim

To implement **Logistic Regression** for binary classification using the **gradient descent** optimization technique.

Algorithm

- Initialize the training data and class labels.
- Initialize weight and bias to zero.
- Calculate the linear output.
- Apply the **sigmoid function** to obtain probability.
- Calculate the error and gradients.
- Update weight and bias using the learning rate.
- Repeat for the specified number of epochs.
- Classify the output using a threshold of **0.5**.
- Calculate and display accuracy.

Input

- I. Dataset: $X = [1, 2, 3, 4, 5, 6]$
- II. Classes: 0, 0, 0, 1, 1, 1
- III. Learning rate = 0.1
- IV. Epochs = 1000

Output

Sigmoid Outputs:

1 → 0.001 → Class 0
2 → 0.009 → Class 0
3 → 0.102 → Class 0
4 → 0.898 → Class 1
5 → 0.991 → Class 1
6 → 0.999 → Class 1

Accuracy: 100%

Program & Output

```
import math
X = [1,2,3,4,5,6]
y = [0,0,0,1,1,1]
lr = float(input("Enter learning rate: "))
w, b = 0, 0
def sigmoid(z):
    return 1/(1+math.exp(-z))
for _ in range(1000):
    dw = db = 0
    for x, t in zip(X,y):
        p = sigmoid(w*x+b)
        dw += (p-t)*x
        db += p-t
    w -= lr*dw/len(X)
    b -= lr*db/len(X)
print("\nSigmoid Outputs:")
for x in X:
    p = sigmoid(w*x+b)
    print(x, round(p,3), "Class:", int(p>=0.5))
correct = sum(int(sigmoid(w*x+b)>=0.5)==t for x,t in zip(X,y))
print("Accuracy:", correct/len(X)*100, "%")

... Enter learning rate: 0.1
```

```
... Enter learning rate: 0.1
```

```
Sigmoid Outputs:
1 0.018 Class: 0
2 0.093 Class: 0
3 0.359 Class: 0
4 0.755 Class: 1
5 0.944 Class: 1
6 0.989 Class: 1
Accuracy: 100.0 %
```

Result

The Logistic Regression model was successfully implemented using **gradient descent** and **sigmoid activation**, achieving **100% accuracy** on the given dataset.